SG-ALNS: Segment-Level Graph RL for ALNS Configuration in the VRPTW

Duong Q. Vo
Faculty of Information Technology
Vinh University
Nghe An, Vietnam
235748020110196@vinhuni.edu.vn

Sang X. Tran*
Institute of Information Technology
and Digital Transformation, Vinh University
Nghe An, Vietnam
sangtx@vinhuni.edu.vn

Uyen T. Nguyen*
Faculty of Information Technology
Vinh University
Nghe An, Vietnam
uyennt@vinhuni.edu.vn

***Abstract*—Reinforcement learning (RL) is increasingly used to control Adaptive Large Neighbourhood Search (ALNS), and its gains are usually attributed to state-dependent control. We test this attribution on the vehicle routing problem with time windows (VRPTW) with SG-ALNS, a segment-level graph RL controller that jointly selects the operators, the destroy scale and the annealing temperature, and can fall back to the classical roulette wheel. Six controls, namely policy inspection with counterfactual states, training-seed replication, tuned baselines, a dynamic-headroom search, equal run time, and transfer across budgets and sizes, show that SG-ALNS learns an effective configuration, not a controller: 17 of 20 trained policies select a single action and ignore counterfactual changes of their state, cheaply tuned static configurations are statistically equivalent in fleet size, and the fleet reduction over the roulette wheel (418.0 vs. 421.4 vehicles on Solomon) does not survive a correction for multiple comparisons. These results show which controls are needed before the gains of RL are attributed to adaptive control.**

***Index Terms*—reinforcement learning, adaptive large neighbourhood search, algorithm configuration, evaluation methodology, vehicle routing problem with time windows**

I. Introduction

The vehicle routing problem with time windows (VRPTW) asks for minimum-cost routes that serve every customer within its time window. It is NP-hard, and Adaptive Large Neighbourhood Search (ALNS) [1], [2] is one of the most widely used metaheuristics for it [3]. At every iteration, a *destroy* operator removes q customers from the incumbent, a *repair* operator reinserts them, and a simulated-annealing (SA) criterion decides acceptance; a roulette wheel (RW) biases operator selection towards recently successful operators. A meta-analysis [4], however, found that this adaptive layer improves solution quality by only about 0.14% on average. This makes ALNS a natural target for learned *algorithm control* [5], also known as parameter control [6] or dynamic algorithm configuration (DAC) [7], [8]. Recent works train deep reinforcement learning (RL) agents to control ALNS at every iteration [9]–[12] and report gains over RW.

These agents share three limitations. First, to keep the Markov decision process (MDP) tractable, they are trained on short episodes (20 decisions in GRLOS [11], 100 iterations in DR-ALNS [10]) or on runs shorter than at deployment (1,000 vs. up to 10,000 iterations in DRLH [9]), and Johnn et al. [11] report degradation outside the training horizon. Second, most of them fix the destroy scale and the temperature, although both strongly affect ALNS [1], [13]; DR-ALNS [10] and recent variants [14], [15] also learn them. Third, to our knowledge, none of them can fall back to the RW rule as one of its actions.

More fundamentally, it is rarely checked *why* the agent wins: does it exploit the search state, or has it merely found a better static configuration? Control subsumes configuration, so a controller can win without being adaptive; DAC studies therefore compare learned policies with the best static configuration [7], [16]. In RL-controlled ALNS, Reijnen et al. [10] compare with an ALNS tuned by SMAC, but whether the learned policies actually depend on the state, and how they vary across training runs, has not been reported.

We study this question with SG-ALNS (Segment-level Graph ALNS), a controller designed to be favourable to RL. Following the DAC formulation, in which a policy reconfigures the algorithm at fixed intervals [7], one decision configures the next K iterations, so that proximal policy optimisation (PPO) can be trained on complete ALNS runs. Each decision jointly selects the operators, the destroy scale and the temperature from a graph state with dynamic time-window features, and the action space contains the RW rule. Our contributions are:

- **A segment-level MDP for ALNS** on the VRPTW, used as a deliberately favourable test bed rather than as a new principle: it adopts interval-based decisions from DAC [7] and joint control of operators, destroy scale and temperature from DR-ALNS [10], and adds full-trajectory training, a dynamic time-window graph state and an *expert-inclusive* action space.
- **An evaluation protocol with six controls** for RL-controlled ALNS. Static baselines and Friedman–Nemenyi tests are established practice in DAC and algorithm benchmarking [16], [17]; we combine them with a policy-collapse diagnostic, training-seed replication, tuned static *and* tuned ALNS-RW baselines with a tuning budget smaller than one training run, a dynamic-headroom test, a time-matched comparison, and transfer across budgets and instance sizes up to $n = 400$, all eval-

*Corresponding authors: Sang X. Tran (sangtx@vinhuni.edu.vn) and Uyen T. Nguyen (uyennt@vinhuni.edu.vn).

uated with hierarchical two-sided tests and equivalence tests.

- **Evidence that learned ALNS control on the VRPTW is, in our setting, learned configuration**: 17 of the 20 trained policies choose a constant action on the instances and budgets we examined and ignore counterfactual changes of their state, while the exceptions are worse or, in one training seed, a time-dependent schedule no better than a tuned constant; seed-to-seed differences dominate, tuned configurations are equivalent in fleet size, time-dependent schedules offer small headroom that a direct search finds but the policies do not exploit, and transfer follows the training distribution rather than any adaptation.

Table I summarises the six controls and their outcome; Section V details them.

II. Related Work

ALNS. ALNS [1], [2] combines destroy operators (e.g. Shaw [18] and string [19] removals), repair operators, an acceptance criterion [13] and a roulette wheel. Unlike neural LNS [20], [21], we keep classical operators and learn how to schedule them.

RL-controlled ALNS. Kallestad et al. [9] (DRLH) replace the adaptive layer with a deep RL hyper-heuristic that selects a heuristic at every iteration; it is trained with PPO on complete 1,000-iteration runs under a fixed SA schedule. DR-ALNS [10], the closest work to ours, also sets the destroy severity and the SA temperature at every iteration, from seven flat features and 100-iteration training episodes; joint control of operators and parameters is therefore not new. GRLOS [11] encodes the solution with a graph neural network (GNN) and selects operators with a deep Q-network, using a fixed destroy scale and short training horizons. PPO-ALNS [12] integrates PPO-based operator selection into ALNS for the VRPTW. Concurrently, Cherrered et al. [14] let PPO choose the destroy operator, the destruction degree and the temperature for the CVRP (the VRPTW is in progress), Xie and Li [15] apply DR-ALNS-style control to pod repositioning, and DAC-ALNS [22] selects operators with a GNN-based dual actor-critic. Our controller thus shares its ingredients with these works. None of them reports the learned actions, their variation across training seeds or an equal-time comparison (Table II).

Configuration versus control. Offline configurators such as irace [23] fix parameter values for an instance distribution, whereas parameter control [6] and DAC [7], [8], often solved by RL [24], adapt them during a run; we bring this comparison to RL-controlled ALNS.

III. Problem and Method

a) VRPTW: Customers $i \in \{1, \dots, n\}$ have demand d_i, service time σ_i and a hard time window $[e_i, l_i]$; identical vehicles of capacity Q start and end at depot 0, and travel times equal unrounded Euclidean distances c_{ij}. Service at i starts at $b_i = \max\{e_i,\, b_p + \sigma_p + c_{pi}\} \le l_i$, where p is the predecessor of i. As is customary, we minimise the number of vehicles NV, then the total distance TD, via $f = 10^4\,\mathrm{NV} + \mathrm{TD}$; all solutions are re-checked by an independent feasibility checker.

b) Underlying ALNS: All methods share the same ALNS: Solomon's I1 construction [25], a vehicle-reduction phase of $10n$ iterations [1], and a main phase of T destroy–repair iterations with SA acceptance (τ_0 accepts a deterioration of 5% of the initial distance with probability 0.5; $\tau_t = \tau_0 \alpha^t$ with $\alpha = 0.01^{1/T}$); a repair that cannot reinsert all customers is discarded. The portfolio has six destroy operators (random, worst, Shaw, string, route and waiting-time) and three repair operators (greedy, regret-3 and noisy greedy). Insertion feasibility is checked in $O(1)$ per position with forward time slack [26], in Numba kernels. After each segment (100 iterations in ALNS-RW, K inside SG-ALNS), RW updates the weight of operator o as $w_o \leftarrow \max\{(1-\rho)w_o + \rho\,\psi_o/u_o, 10^{-3}\}$, where ψ_o is the total score of o in the segment (33, 9 or 13 per use) and u_o its number of uses ($\psi_o/u_o = 0$ if $u_o = 0$), with $\rho = 0.1$; the normalised weights give the destroy and repair distributions $(p_D, p_R) = \mathrm{RW}(\boldsymbol{w})$; the destroy scale is drawn as $q \sim \mathcal{U}[0.05n, 0.30n]$.

c) Segment-level MDP: An episode is one complete main phase, split into $H = T/K$ segments ($K = 50$). At the start of each segment, the agent observes the state s_h and selects a factorised action $a_h = (a^1, a^2, a^3)$. The component $a^1 \in \{0, \dots, 18\}$ selects either a *lead* destroy–repair pair, used with probability 0.7 while the other operators share the remaining probability uniformly, or ($a^1 = 18$) the current RW distribution, whose weights keep being updated; this defines (p_D, p_R). The component a^2 sets the destroy range $\mathcal{Q}$: $q = \kappa n$ with $\kappa \in \{5, 10, 20, 30\}\%$ or the default $q \sim \mathcal{U}[5, 30]\%\, n$, and a^3 selects a deviation $m \in \{0.3, 1, 3\}$ *around* the cooling schedule, $\tau = m\tau_0\alpha^t$. The action $(18, \mathrm{default}, 1)$ reproduces ALNS-RW; we call this action space *expert-inclusive*, in the spirit of DAC policies that select among existing heuristics [24]. A headroom study on generated instances, run before training, motivates it: no fixed lead pair or destroy scale dominates RW (Table S4). The reward

$$r_h = 100\,\frac{\mathrm{TD}^\star_{h-1} - \mathrm{TD}^\star_h}{\mathrm{TD}^\star_0} + \lambda\,(\mathrm{NV}^\star_{h-1} - \mathrm{NV}^\star_h), \qquad \lambda = 5, \tag{1}$$

uses the best solution $x^\star$ after each segment; since consecutive terms cancel, the undiscounted return of an episode equals the final hierarchical improvement (with the discount $\gamma = 0.99$ used in training, early improvements weigh slightly more). Unlike f, which orders all our solutions lexicographically (supplement), the reward trades one vehicle against 5% of the initial distance; it is a shaping choice, and we test its sensitivity to λ (Section V-B).

d) State and policy: The state graph contains the 10 nearest-neighbour arcs of each node and the arcs of the incumbent. Each node has 6 static features (coordinates, demand, time window and service time) and 6 dynamic ones (waiting time, slack, forward time slack, load ratio, route position and a routed flag), all scaled to $[0, 1]$ as detailed in the supplement. A global vector holds the progress t/T, the temperature ratio τ/τ_0, a stagnation counter, the gaps of

TABLE I
THE SIX CONTROLS AT A GLANCE

Control	Result
(1) Collapse	17 of 20 policies use one action in all decisions (Fig. 3); 0.0% of the counterfactual queries change the action of a collapsed policy (Table V)
(2) Training seeds	CNV 419.3±0.9 over 5 seeds; 4 different actions; 2 of 5 use significantly fewer vehicles than ALNS-RW
(3) Tuned baselines	vs. tuned static: ΔCNV −0.6 [−2.0, +0.8], p=0.581 (Table IV)
(4) Headroom	best two-phase schedule 0.81 vs. best constant 0.70 (p=0.51); per-class oracle saves 0.6 vehicles over the best constant
(5) Equal run time	vs. tuned static at T=10k: ΔCNV +0.2 [−1.2, +1.6], p=1.000
(6a) Budget	vs. tuned static, T=1k: ΔCNV −0.7 (p=0.518); T=20k: ΔCNV +1.0 (p=0.336)
(6b) Size n=400	vs. ALNS-RW: SG-ALNS ΔTD +2.37% ($p < 0.001$), SG-ALNS-C −1.06% ($p < 0.001$) on equal-fleet instances

Solomon, $T = 5{,}000$ unless stated; Δ = SG-ALNS minus baseline, two-sided tests; details in Tables III and IV.

TABLE II
POSITIONING AGAINST RL-CONTROLLED ALNS WORKS

Work	Decision	Graph	Dyn. TW	Learns q	Learns τ	Full-run training	RW action
DRLH [9]	iteration	–	–	–	–	✓‡	–
DR-ALNS [10]	iteration	–	–	✓	✓	–†	–
ALNS-RL [14]	n/r	–	✓	✓	✓	n/r	–
GRLOS [11]	iteration	✓	–	–	–	–	–
SG-ALNS (ours)	segment	✓	✓	✓	✓	✓§	✓

✓: yes; –: no; n/r: not reported (as reported in the cited papers).
† 100-iteration episodes, evaluated on 100–200 iterations.
‡ complete 1,000-iteration runs, deployed for up to 10,000 iterations.
§ complete 2,500-iteration runs, deployed for 5,000 (1,000–20,000 in Section V-C).

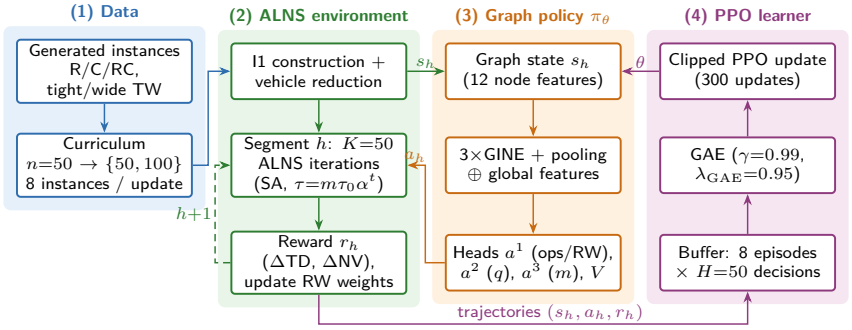


Fig. 1. Training framework of SG-ALNS: generated instances (1) feed complete ALNS episodes split into segments of $K = 50$ iterations (2); before each segment the graph policy maps the state s_h to an action a_h (3); PPO updates θ from 8 parallel episodes (4). Algorithm 1 details one episode.

the current and initial solutions to the best one, the fleet ratio and the operators' success rates in the last segment. Three residual graph isomorphism layers with edge features (GINE) [27] of width 64, followed by mean and max pooling, feed three conditionally independent categorical heads, so that $\log \pi(a \mid s) = \sum_k \log \pi_k(a^k \mid s)$, and a value head (58k parameters). Output biases are initialised to favour the expert action, so that learning starts near ALNS-RW (a neutral initialisation is tested in Section V-B).

e) Training (Fig. 1): Each of 300 PPO [28] updates runs Algorithm 1 with $T = 2{,}500$ on 8 generated instances in parallel and uses generalised advantage estimation (GAE) [29] ($\gamma = 0.99$, $\lambda_{\mathrm{GAE}} = 0.95$, clip 0.2, Adam with learning rate $3 \cdot 10^{-4}$, entropy coefficient annealed from 0.01 to 0.001). Instances follow Solomon's design, with $n = 50$ for the first 40% of the updates and $n \in \{50, 100\}$ afterwards. No benchmark instance is used for training or model selection. Training runs are complete but shorter than at deployment ($T = 2{,}500$ vs. 5,000): as t/T and the cooling schedule are normalised by T, the policy sees the same relative trajectory, yet its advantage over RW appears only after 2,500 iterations (Fig. 2a). We evaluate the final checkpoint of 5 training seeds (seed 0, used for all ablations, was designated beforehand). One training run takes 21–23 minutes on 8 cores.

f) Context-aware variant (SG-ALNS-C): SG-ALNS observes neither n nor T, so it cannot condition q on the

Algorithm 1: SG-ALNS: one episode (actions sampled in training, greedy at deployment)

Input: instance I, policy π_θ, segment length K, budget T
Output: best solution $x^\star$; trajectory $(s_h, a_h, r_h)_{h=1}^{H}$

$x \leftarrow \mathrm{VehicleReduction}(\mathrm{SolomonI1}(I))$;
$x^\star \leftarrow x$; $\boldsymbol{w} \leftarrow \mathbf{1}$; $t \leftarrow 0$; $H \leftarrow T/K$;
for $h = 1, \ldots, H$ **do** `// one decision per segment`
  $s_h \leftarrow \mathrm{GraphState}(x, x^\star, t/T, \tau/\tau_0, \text{success rates})$;
  $a_h = (a^1, a^2, a^3) \sim \pi_\theta(\cdot \mid s_h)$;
  if $a^1 = 18$ **then** $(p_D, p_R) \leftarrow \mathrm{RW}(\boldsymbol{w})$;
  else $(p_D, p_R) \leftarrow$ lead pair a^1 w.p. 0.7;
  $\mathcal{Q} \leftarrow$ destroy range of a^2; $m \leftarrow m(a^3)$;
  for $k = 1, \ldots, K$ **do**
    $\tau \leftarrow m\,\tau_0\alpha^t$; $o_D \sim p_D$; $o_R \sim p_R$; $q \sim \mathcal{Q}$;
    $x' \leftarrow o_R(o_D(x, q))$; $t \leftarrow t + 1$;
    if $f(x') < f(x^\star)$ **then** $x^\star \leftarrow x'$;
    if $\mathrm{AcceptSA}(x', x, \tau)$ **then** $x \leftarrow x'$;
  $r_h \leftarrow$ Eq. (1); update $\boldsymbol{w}$ from the segment's scores;
return $x^\star$ and $(s_h, a_h, r_h)_{h=1}^{H}$;

instance size. SG-ALNS-C appends $n/200$ and $T/5{,}000$ to the global vector and is trained on $n \in \{50, 100, 200\}$ with $T \in \{1{,}000, 2{,}500, 5{,}000\}$ drawn per update (33 minutes), a setting that favours instance-dependent behaviour.

IV. Experimental Setup

a) Benchmarks: We use the 56 Solomon instances ($n = 100$) [25] and the 60+60 Gehring–Homberger instances with $n = 200$ and $n = 400$ [30]. For Solomon, the reference is the hierarchical best-known solution (BKS) table of SINTEF, cross-checked against the CVRPLIB solutions (identical on all 19 instances with the BKS fleet; none of our runs dominates a BKS). For Gehring–Homberger, whose hierarchical BKS are not available in machine-readable form, the reference is the *best solution found* over all our runs.

b) Baselines: ALNS-Uni samples operators uniformly; ALNS-RW is the classical roulette wheel of Section III. **Tuned ALNS-RW** is the best of 36 RW configurations (update period, reaction factor ρ, maximum destroy scale $q_{\max}$ and temperature deviation m; selected: K=100, ρ=0.3, $q_{\max}$=40%, m=1). **Tuned static** is the best of 30 constant actions of the SG-ALNS action space: the RW action and String/Regret-3, the best lead pair of the pre-training headroom study (Table S4), each with all destroy scales and temperatures (selected: String/Regret-3, $q = 30\%$, $m = 0.3$). The grid thus contains the action later learned by seed 0, but was fixed before training. Both baselines are selected on 24 *generated* instances (3 seeds, $T = 5{,}000$) by the score $5\Delta\mathrm{NV} + 100\Delta\mathrm{TD}/\mathrm{TD}$ against ALNS-RW (the weighting of Eq. (1)). This takes 2,232 and 2,664 runs of 5,000 iterations, i.e. 25 and 37 CPU-minutes (about 3 and 5 minutes on 8 cores), whereas one training run (2,400 episodes of 2,500 iterations plus network updates) occupies 8 cores for 21–23 minutes. The tuning instances are disjoint from the training and validation instances, and the benchmarks serve only as test sets; Table S14 relates the tuning budget to quality on held-out tuning instances, which saturates at about 30 random constant actions (score +0.47). As a check against a too narrow grid, we also tuned over all 285 constant actions (20,592 runs, 302 CPU-minutes). The **iteration-level baselines** make one PPO decision per iteration over the 18 operator pairs, trained on 100-iteration episodes as in DR-ALNS [10], with an MLP encoder (PPO-ALNS-like [12], 2,400 episodes) or our GNN encoder (GRLOS-like [11]; stopped after 1,368 episodes or 57 minutes, 2.6× the training time of SG-ALNS).

c) Protocol: Solomon runs use 5 seeds per instance (3 for the iteration-level baselines and the budget study), Gehring–Homberger runs 3. We report cumulative vehicles (CNV) and distance (CTD), the TD gap to the reference over runs with the reference NV, and the primal integral (PI) of f, in % of the reference. Because f mixes two scales, we never rank-test it. The unit of every test is the instance: the runs (seeds) of a method on an instance are averaged first. Pairwise comparisons are hierarchical and two-sided: the difference in CNV (bootstrap 95% CI, Wilcoxon signed-rank test on per-instance mean NV), then the relative TD difference on the instances where both methods have the same mean NV. Equivalence is tested with TOST ($\alpha = 0.05$) using margins fixed beforehand: 0.1 vehicles per instance and 0.2% of distance. The fleet margin, one vehicle on every tenth instance, is practically negligible but exceeds the training-seed standard deviation of SG-ALNS (0.9 vehicles in total), so we also report equivalence at half the margin. W/T/L counts the instances on which SG-ALNS has a lower, equal (identical solutions) or higher mean f, with a two-sided sign test (Holm-corrected over the baselines of Table III). Friedman–Nemenyi tests [17] cover the six methods run with 5 seeds. Each run uses one core of an Apple M1 (times include vehicle reduction and inference). Tables S1–S17, Figs. S1–S4 and implementation details are in the supplement. The tuned baselines, seeds 1–4, SG-ALNS-C, the time-matched study, the $n = 400$ set and the analyses of Table V and Tables S14–S17 were added after we observed policy collapse; none uses benchmark data for selection. Code, trained models, raw results and the BKS table are available at https://github.com/XXXX/sg-alns.

V. Results

A. Main Results on Solomon

Table III reports $T = 5{,}000$. The pre-designated seed of SG-ALNS uses 418.0 vehicles against 421.4 for ALNS-RW (BKS: 405), with the lowest gap (1.48%) and primal integral. In the hierarchical test (Table IV) the fleet reduction is significant on its own ($\Delta\mathrm{CNV} = -3.4$, 95% CI $[-6.2, -1.2]$, $p = 0.012$) but not after Holm correction over the comparisons of Table IV ($p_{\mathrm{Holm}} = 0.081$), and it amounts to 0.06 vehicles per instance. This is the best of the 5 training seeds: averaged over all of them, SG-ALNS uses 419.3±0.9 vehicles. The W/T/L on f (25/16/15) is not significant ($p = 0.615$): the 16 ties are C1/C2 instances solved identically by all methods, and the vehicle gains come from a few R1, R2 and RC1 instances (Fig. S3a). SG-ALNS pulls ahead of all baselines after about 2,500 iterations, the end of its training horizon

TABLE III
SG-ALNS AGAINST THE BASELINES ON SOLOMON ($T = 5{,}000$)

Method	CNV	CTD	Gap % (runs)	PI %	Time s	W/T/L	p
BKS	405	57,187	0.00	–	–	–	–
SG-ALNS (seed 0, pre-designated)	418.0	57,188	1.48 (215)	5.05	2.15	–	–
SG-ALNS, 5 training seeds (mean±sd)	419.3±0.9	57,211	1.77±0.29	5.12	2.41	–	–
ALNS-RW	421.4	57,115	1.86 (201)	5.58	1.09	25/16/15	0.615
Tuned ALNS-RW	419.8	57,144	1.61 (206)	5.30	1.44	25/17/14	0.541
ALNS-Uni	421.0	57,352	2.09 (201)	5.44	1.10	35/16/5	<0.001
Tuned static	418.6	57,190	1.69 (212)	5.07	0.97	25/16/15	0.615
Tuned static, all 285 actions	419.0	57,017	1.40 (210)	5.20	2.34	18/16/22	1.000
SG-ALNS-C	422.0	57,512	2.44 (195)	5.56	1.71	34/16/6	<0.001
SG-ALNS (sampled)	418.4	57,190	1.53 (213)	4.95	1.44	21/16/19	1.000
Iteration-level (GNN / MLP)*	425.3	57,424	2.63 (111)	5.83	18.8 / 10.7	34/15/7	<0.001

5 seeds (*3 seeds, compared on common seeds). Gap: TD gap to the BKS over runs with the BKS fleet (number of runs). W/T/L: instances where SG-ALNS has a lower/equal/higher mean f; p: two-sided sign test, Holm-corrected. *Our re-implementations (100-iteration training episodes); both return identical solutions.

TABLE IV
HIERARCHICAL TWO-SIDED COMPARISON WITH THE STRONGEST BASELINES (SOLOMON, $T = 5{,}000$)

vs.	ΔCNV [95% CI]	+/−	p	p_{H}	Eq.	ΔTD % [95% CI] (n)	p	Eq.
ALNS-RW	−3.4 [−6.2, −1.2]	8/0	0.012	0.081	–	−0.30 [−0.74, +0.05] (48)	0.246	–
Tuned ALNS-RW	−1.8 [−3.8, −0.2]	7/2	0.020	0.123	✓	−0.06 [−0.26, +0.16] (47)	0.263	–
Tuned static	−0.6 [−2.0, +0.8]	3/1	0.581	1.000	✓	−0.22 [−0.63, +0.11] (52)	0.423	–
Tuned static (285)	−1.0 [−2.4, +0.2]	5/2	0.061	0.243	✓	+0.11 [−0.16, +0.35] (49)	0.122	–
ALNS-RW, T=10k	−1.8 [−3.6, −0.4]	6/1	0.031	0.156	✓	+0.07 [−0.35, +0.39] (49)	0.102	–
Tuned ALNS-RW, T=10k	−0.8 [−2.2, +0.2]	3/1	0.141	0.422	✓	+0.29 [+0.10, +0.48] (52)	0.005	–
Tuned static, T=10k	+0.2 [−1.2, +1.6]	2/3	1.000	1.000	✓	+0.16 [−0.18, +0.43] (51)	0.031	–

Δ: SG-ALNS minus baseline (negative: SG-ALNS better), 95% CI; +/−: instances with fewer/more vehicles; p: unadjusted, p_{H}: Holm over the 7 rows; Eq.: TOST equivalence; "T=10k": baseline at equal run time.

(Fig. 2a). Our iteration-level agents are worse than SG-ALNS and even than ALNS-RW (425.3 vehicles), 5–9× slower, and stall after 1,000 iterations. Trained on 100-iteration episodes, both always apply Worst/Regret-3 (Fig. 3a) and return identical solutions. Even at $T = 200$, close to their training horizon, they only match ALNS-RW (428.3 vs. 428.3 vehicles), whereas SG-ALNS uses 425.0 (Table S16): our iteration-level agents learned no useful control in either regime, so the comparison does not show that segment-level decisions are better in general. Against the two *tuned* baselines, SG-ALNS is not better in a practically relevant sense: $\Delta\mathrm{CNV} = -1.8$ against tuned ALNS-RW ($p = 0.020$) and -0.6 against tuned static ($p = 0.581$), and in both cases TOST shows equivalence within 0.1 vehicles per instance: a difference can be real and still smaller than this margin. At half the margin, tuned static remains equivalent ($p = 0.002$; all 285 actions: $p = 0.007$), but tuned ALNS-RW does not ($p = 0.145$). No distance difference is significant. Tuning over all 285 constant actions instead selects Worst/Regret-3, q=30%, m=3, which is equivalent as well ($\Delta\mathrm{CNV} = -1.0$, $p = 0.061$). In the Friedman–Nemenyi test, SG-ALNS, both tuned baselines and ALNS-RW form one group (Fig. S4a).

B. Learned Control or Learned Configuration?

a) Collapse: In Fig. 3a, 17 of the 20 trained policies choose *one* action in 100% of their decisions on Solomon, on every instance. This includes SG-ALNS-C, which observes n and T yet selects the same action at $n = 100$, 200 and 400 and at every budget (Table S11). Our best policy, seed 0, always selects RW weights, q=30%, m=3: it has learned a re-parametrised ALNS-RW, not a controller. Of the three exceptions, training seed 3 and the neutral-prior policy are discussed below; the third, without a^1, alternates between the default destroy range and $q = 20\%$ (8.7 switches per run) and uses more vehicles than SG-ALNS ($\Delta\mathrm{CNV} = -3.6$, $p = 0.007$). Entropy collapses during training (Fig. S4b), and sampling actions does not change the result ($\Delta\mathrm{CNV} = -0.4$, $p = 0.516$). As constancy on test instances does not prove state independence, we probed the policies (Table V): for the 12 collapsed policies we probed, 0.0% of about 30,000 counterfactual queries per policy change the greedy action, whether the progress, temperature, stagnation and success features are redrawn at random, the dynamic node features are permuted or zeroed, or the graph is replaced by that of another instance. The 3 policies that use several actions do react (4.5–57.0% of queries); two of them, the variant without a^1 and the neutral-prior policy below, perform worse, and the third, training seed 3, is discussed next.

b) Training seeds: If the policy encodes a constant, different training runs may find different ones, and they do: the 5 seeds learn 4 different actions (Fig. 3a), and 2 of the 5 use significantly fewer vehicles than ALNS-RW (unadjusted;

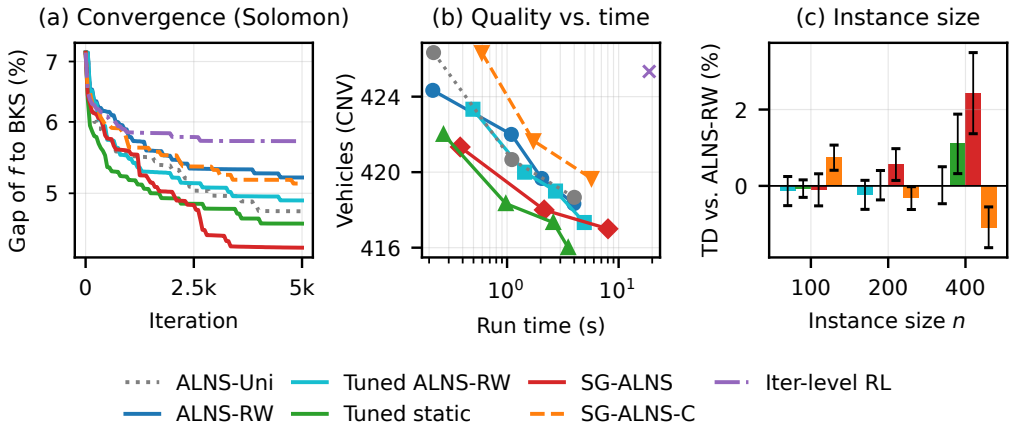


Fig. 2. (a) Mean gap of the best-so-far f to the BKS during the main phase (Solomon, 3 seeds; f is dominated by the fleet size). (b) Cumulative vehicles on Solomon vs. mean run time for $T \in \{1, 5, 10, 20\} \cdot 10^3$ (3 seeds); points further down-left are better. (c) Mean relative distance to ALNS-RW on instances where both use the same mean fleet (95% confidence intervals) for $n = 100$ (Solomon), 200 and 400 (Gehring–Homberger).

TABLE V
STATE-SENSITIVITY PROBE

Policy	Natural	Global	Dynamic	No dyn.	Other
SG-ALNS seed 0	0.0 / 0.088	0.0 / 0.028	0.0 / 0.004	0.0 / 0.035	0.0 / 0.014
seed 1	0.0 / 0.129	0.0 / 0.028	0.0 / 0.011	0.0 / 0.151	0.0 / 0.055
seed 2	0.0 / 0.007	0.0 / 0.001	0.0 / 0.001	0.0 / 0.015	0.0 / 0.002
seed 3	31.6 / 0.235	57.0 / 0.097	6.4 / 0.017	23.5 / 0.044	21.8 / 0.037
seed 4	0.0 / 0.008	0.0 / 0.002	0.0 / 0.002	0.0 / 0.014	0.0 / 0.007
SG-ALNS-C	0.0 / 0.098	0.0 / 0.021	0.0 / 0.011	0.0 / 0.038	0.0 / 0.020
neutral prior, seed 0	61.8 / 0.211	18.1 / 0.083	6.8 / 0.018	34.6 / 0.120	53.3 / 0.079
MLP encoder	0.0 / 0.165	0.0 / 0.081	0.0 / 0.077	0.0 / 0.070	0.0 / 0.054
w/o expert	0.0 / 0.086	0.0 / 0.024	0.0 / 0.014	0.0 / 0.070	0.0 / 0.036
w/o a^1	92.2 / 0.079	7.7 / 0.048	4.5 / 0.008	37.0 / 0.025	18.1 / 0.030
$\lambda = 1$	0.0 / 0.019	0.0 / 0.003	0.0 / 0.004	0.0 / 0.029	0.0 / 0.015
$\lambda = 25$	0.0 / 0.027	0.0 / 0.006	0.0 / 0.001	0.0 / 0.013	0.0 / 0.016
neutral prior, seed 1	0.0 / 0.122	0.0 / 0.030	0.0 / 0.007	0.0 / 0.095	0.0 / 0.038
neutral prior, seed 2	0.0 / 0.199	0.0 / 0.051	0.0 / 0.006	0.0 / 0.010	0.0 / 0.046
seed 0, 5× training	0.0 / 0.222	0.0 / 0.052	0.0 / 0.007	0.0 / 0.083	0.0 / 0.047

12 Solomon instances (two per class, $T = 5{,}000$). Each cell: share of queries whose greedy action differs from the action actually taken (%) / mean total-variation distance between the joint action distributions. Natural: visited states vs. the first state of the run; global: progress, temperature, stagnation, gaps, fleet ratio and operator success rates redrawn at random (16 per decision); dynamic: dynamic node features permuted across nodes (4); no dyn.: dynamic node features set to 0; other: graph of another instance (4).

Table S3). Seed 3 is the only one that uses its state: it alternates between Worst/Regret-3, q=30%, m=1 and Worst/Regret-3, q=30%, m=3, raising the temperature as the search progresses (the hotter action takes 0% of the decisions in the first 1,000 iterations and 85% in the last), much like the best two-phase schedule below. This learned control, however, is equivalent in fleet size to tuned static (419.2 vehicles; $\Delta\text{CNV} = +0.6$, $p = 0.581$, TOST). Likewise, all 5 variants without a^1, a^2, a^3, dynamic features or message passing use significantly more vehicles than SG-ALNS (Table S7), but each is another training run that converged to another constant; the variants without dynamic features and without message passing even learn the action of seed 1 and reproduce its results exactly (Fig. 3a). These ablations thus measure seed-to-seed variation, not the value of the encoder or the features. Conversely, if the controller is a constant, a constant tuned offline should do as well, and it does: tuned static reaches 418.6 vehicles at less than half the run time, and tuned ALNS-RW ($q_{\max} = 40\%$, $\rho = 0.3$) moves ALNS-RW from 421.4 to 419.8 vehicles.

c) Headroom for control: A collapse may reflect a weak learner rather than an absence of useful control, so we searched directly for time-dependent control: all 90 two-phase schedules (one action per half of the main phase) over the 10 best constant actions, tuned like the baselines (Table S6). The best schedule (String/Regret-3, 20%, m=0.3 → String/Regret-3, 30%, m=3) scores 0.81 against 0.70 for the best constant ($p = 0.51$); only 11 schedules beat it, and one selected on two seeds is worse on the third. On Solomon, the schedule uses as many vehicles as the constant ($\Delta\text{CNV} = +0.4$, $p = 0.461$) but shorter routes ($\Delta\text{TD} = -0.23\%$, $p = 0.012$, 52 instances). Time-dependent headroom thus exists; it is small, and a direct search finds it while none of our policies does.

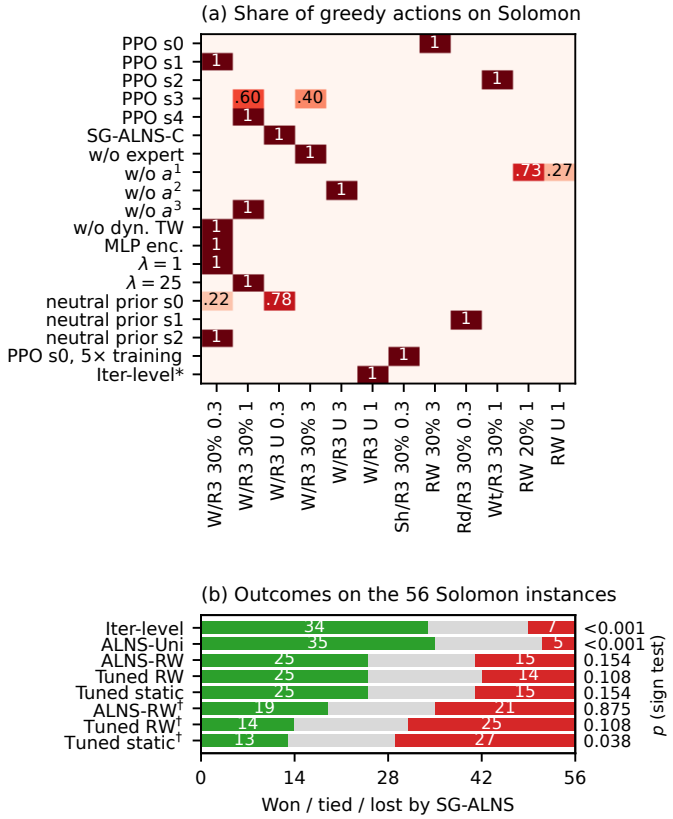


Fig. 3. (a) Share of each greedy action over all decisions on Solomon for the 20 trained policies (s0, s1, …: training seeds; the two iteration-level agents share a row). Actions are written lead pair, q, m with W = Worst, Wt = Wait, Sh = Shaw, Rd = Random, R3 = Regret-3, U = $\mathcal{U}[5, 30]$%; "RW 30% 3" is ALNS-RW with $q = 30\%$ and a threefold temperature. (b) Instances won, tied and lost by SG-ALNS on f, with unadjusted two-sided sign-test p; †baselines run with $T = 10{,}000$ (equal time).

Instance-dependent control has even less room: choosing, with hindsight, the best of the 15 best constants separately for each Solomon class saves only 0.6 vehicles over the best single constant (417.4 vs. 418.0), and choosing per class on the generated instances does not help (419.4). PPO hyperparameters do not appear to cause the collapse: with a constant entropy coefficient of 0.01 and neutral initial biases, 2 of 3 training seeds again select a single action, and the third uses 2 actions that differ only in q and has longer routes than ALNS-RW (ΔTD $= +0.51\%$, $p = 0.001$). Nor does training length: trained five times longer (1,500 updates), seed 0 collapses to Shaw/Regret-3, q=30%, m=0.3 (419.6 vehicles) and ignores counterfactual states (Table V). Nor does the reward weight: retrained with $\lambda = 1$ and $\lambda = 25$, seed 0 again collapses to a single action (Worst/Regret-3, q=30%, m=0.3 and Worst/Regret-3, q=30%, m=1; Table S15).

C. Equal Time, Budgets and Instance Size

a) Time-matched comparison: SG-ALNS needs 2.15 s per run against 1.09 s for ALNS-RW; features and inference take only 3–4% of it (Fig. S3b), the rest depends on the selected operators and temperature (tuned static, also with $q = 30\%$, needs 0.97 s). At equal run time, tuned static lies at or below SG-ALNS except at the shortest budget (Fig. 2b). Giving the baselines $T = 10{,}000$ roughly equalises run time (2.04–2.71 s, slightly favouring the tuned baselines). SG-ALNS then still uses fewer vehicles than ALNS-RW (ΔCNV $= -1.8$, $p = 0.031$, $p_{\text{Holm}} = 0.156$), but it is equivalent in fleet size to both tuned baselines, which find shorter routes (tuned ALNS-RW: ΔTD $= +0.29\%$, $p = 0.005$; tuned static: $+0.16\%$, $p = 0.031$) and win more instances than they lose (Fig. 3b, Table IV).

b) Budgets: Against tuned static, no fleet difference is significant at any budget, but its sign flips from $T = 1{,}000$ (ΔCNV $= -0.7$, $p = 0.518$), the budget closest to the training budget of SG-ALNS, to $T = 20{,}000$ ($+1.0$, $p = 0.336$): each constant is best near the budget it was optimised for (Table S10).

c) Instance size: Fig. 2c transfers all settings unchanged to $n = 200$ and 400 (Table S12). The fixed $q = 30\%$ learned by SG-ALNS becomes harmful at $n = 400$: on equal-fleet instances its distance is +2.37% relative to ALNS-RW ($p < 0.001$). SG-ALNS-C, trained on $n \leq 200$, is the best method at $n = 400$ (gap 1.53% vs. 2.79% for ALNS-RW; ΔTD $= -1.06\%$, $p < 0.001$) and matches the gap of tuned ALNS-RW at $n = 200$ (1.66% vs. 1.66%), yet it is the weakest learned method on Solomon (422.0 vehicles). Its advantage at $n = 400$ lies in C1, C2, R1 and RC1, not in R2 and RC2 (Table S17), and Gehring–Homberger differs from Solomon in more than n, so we attribute it to the training distribution rather than to size alone. As its action is identical at all sizes, this is not adaptation.

D. Discussion

Segment-level RL often finds *good configurations* without hand-tuning, and the best one we tested when trained on the deployment distribution (SG-ALNS-C at $n = 400$). Its policies, however, are (almost) state-independent, and a direct search finds only small time-dependent headroom, consistent with [4]. The differences are also small in practice: among SG-ALNS and the tuned baselines they stay below 0.1 vehicles per instance, while all methods remain about 13 vehicles above the BKS. This does not contradict the gains reported in other settings [9]–[12], but shows which controls they need before being attributed to adaptation. Why the policies collapse remains open: neither the entropy schedule, the initial bias, the reward weight nor five times longer training changes it, leaving the small discrete action space, the discounted reward and the ease of representing a constant as candidate causes. *Threats to validity:* our conclusions concern the VRPTW, our operators and action space, and 5 training seeds, not RL control of ALNS in general. Our base ALNS remains about 13 vehicles above the BKS ($10n$ vehicle-reduction iterations, $T \leq 20{,}000$), and a stronger base may change the headroom; the tuned baselines use grids rather than irace [23], and the iteration-level baselines are our re-implementations rather than the original DR-ALNS or GRLOS code.

VI. Conclusion

We evaluated SG-ALNS, a segment-level RL controller for ALNS on the VRPTW, with rarely applied controls. Its gain over the roulette wheel is fragile across seeds and multiple comparisons; its policies are (almost) state-independent, and the one time-dependent schedule it learned is no better than a tuned constant. In our setting, learned ALNS "control" is, in effect, learned configuration: better results alone do not demonstrate adaptive control, although richer action spaces may leave more room for it. Practitioners can obtain most of the benefit from cheap offline configuration. Future work will test continuous, size-relative actions and electric VRPs.

Disclosure of Interests

The authors have no competing interests to declare that are relevant to the content of this article.

References

[1] S. Ropke and D. Pisinger, "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows," *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.

[2] D. Pisinger and S. Ropke, "A general heuristic for vehicle routing problems," *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007.

[3] S. T. W. Mara, R. Norcahyo, P. Jodiawan, L. Lusiantoro, and A. P. Rifai, "A survey of adaptive large neighborhood search algorithms and applications," *Computers & Operations Research*, vol. 146, p. 105903, 2022.

[4] R. Turkeš, K. Sörensen, and L. M. Hvattum, "Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search," *European Journal of Operational Research*, vol. 292, no. 2, pp. 423–442, 2021.

[5] M. Karimi-Mamaghan, M. Mohammadi, P. Meyer, A. M. Karimi-Mamaghan, and E.-G. Talbi, "Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art," *European Journal of Operational Research*, vol. 296, no. 2, pp. 393–422, 2022.

[6] A. E. Eiben, R. Hinterding, and Z. Michalewicz, "Parameter control in evolutionary algorithms," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 2, pp. 124–141, 1999.

[7] A. Biedenkapp, H. F. Bozkurt, T. Eimer, F. Hutter, and M. Lindauer, "Dynamic algorithm configuration: Foundation of a new meta-algorithmic framework," in *Proceedings of the 24th European Conference on Artificial Intelligence (ECAI)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 325. IOS Press, 2020, pp. 427–434.

[8] S. Adriaensen, A. Biedenkapp, G. Shala, N. Awad, T. Eimer, M. Lindauer, and F. Hutter, "Automated dynamic algorithm configuration," *Journal of Artificial Intelligence Research*, vol. 75, pp. 1633–1699, 2022.

[9] J. Kallestad, R. Hasibi, A. Hemmati, and K. Sörensen, "A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems," *European Journal of Operational Research*, vol. 309, no. 1, pp. 446–468, 2023.

[10] R. Reijnen, Y. Zhang, H. C. Lau, and Z. Bukhsh, "Online control of adaptive large neighborhood search using deep reinforcement learning," in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 34, 2024, pp. 475–483.

[11] S.-N. Johnn, V.-A. Darvariu, J. Handl, and J. Kalcsics, "A graph reinforcement learning framework for neural adaptive large neighbourhood search," *Computers & Operations Research*, vol. 172, p. 106791, 2024.

[12] Z. Wang, Q. Cui, B. Tan, X. Yang, W. Zhou, and X. Huang, "Reinforcement learning-guided adaptive large neighborhood search for vehicle routing problem with time windows," *Journal of Combinatorial Optimization*, vol. 50, p. 35, 2025.

[13] A. Santini, S. Ropke, and L. M. Hvattum, "A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic," *Journal of Heuristics*, vol. 24, no. 5, pp. 783–815, 2018.

[14] A. Cherrered, A. Oulamara, and W. Ramdane Cherif-Khettaf, "Reinforcement learning-enhanced operator selection in adaptive large neighborhood search: Application to CVRP and VRPTW," in *27e Congrès Annuel de la Société Française de Recherche Opérationnelle et d'Aide à la Décision (ROADEF)*, 2026.

[15] L. Xie and H. Li, "Solving the pod repositioning problem with deep reinforced adaptive large neighborhood search," arXiv preprint arXiv:2506.02746, 2025.

[16] T. Eimer, A. Biedenkapp, M. Reimer, S. Adriaensen, F. Hutter, and M. Lindauer, "DACBench: A benchmark library for dynamic algorithm configuration," in *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI)*, 2021, pp. 1668–1674.

[17] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[18] P. Shaw, "Using constraint programming and local search methods to solve vehicle routing problems," in *Principles and Practice of Constraint Programming – CP98*, ser. LNCS, vol. 1520. Springer, 1998, pp. 417–431.

[19] J. Christiaens and G. Vanden Berghe, "Slack induction by string removals for vehicle routing problems," *Transportation Science*, vol. 54, no. 2, pp. 417–433, 2020.

[20] A. Hottung and K. Tierney, "Neural large neighborhood search for the capacitated vehicle routing problem," in *Proceedings of the 24th European Conference on Artificial Intelligence (ECAI)*, 2020, pp. 443–450.

[21] J. K. Falkner, D. Thyssens, and L. Schmidt-Thieme, "Large neighborhood search based on neural construction heuristics," *arXiv preprint arXiv:2205.00772*, 2022.

[22] S. Yu, W. Mao, Z. Wu, and J. Puchinger, "New adaptive mechanism for large neighborhood search using dual actor-critic," arXiv preprint arXiv:2601.11414, 2026.

[23] M. López-Ibáñez, J. Dubois-Lacoste, L. P. Cáceres, M. Birattari, and T. Stützle, "The irace package: Iterated racing for automatic algorithm configuration," *Operations Research Perspectives*, vol. 3, pp. 43–58, 2016.

[24] D. Speck, A. Biedenkapp, F. Hutter, R. Mattmüller, and M. Lindauer, "Learning heuristic selection with dynamic algorithm configuration," in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 31, 2021, pp. 597–605.

[25] M. M. Solomon, "Algorithms for the vehicle routing and scheduling problems with time window constraints," *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.

[26] M. W. P. Savelsbergh, "The vehicle routing problem with time windows: Minimizing route duration," *ORSA Journal on Computing*, vol. 4, no. 2, pp. 146–154, 1992.

[27] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, "Strategies for pre-training graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2020.

[28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[29] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *International Conference on Learning Representations (ICLR)*, 2016.

[30] H. Gehring and J. Homberger, "A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows," in *Proceedings of EUROGEN99*, 1999, pp. 57–64.